Indoor Location Prediction from Wi-Fi Signal Strength: Phase 2, Multitask Deep Learning

Rishi Raj (2401010380), Aditya Samadhiya (2401010037), Yash Yadav (2401010515),
Kaustubh Ranjan Mishra (2401020116), Yash Raj (2401010514)
Advanced Machine Learning, Project 9, Phase 2
Section D, Rishihood University

Abstract—GPS does not work reliably indoors. Wi-Fi fingerprinting converts a scan of received signal strengths (RSSI) into a location estimate. Phase 1 of this project trained linear/logistic regression, k-nearest neighbours and Random Forests on the UJIIndoorLoc dataset (3 buildings, 5 floors, 465 access points, 19,227 training scans). The best model for floor prediction was the per-building Random Forest at 91.3% accuracy; the best for position was k-NN at 8.69 m mean error. All Phase 1 models treated building, floor and coordinates as three separate tasks. Phase 2 trains a single multitask multilayer perceptron with a shared trunk (three hidden layers of 1024, 512 and 256 units, ReLU, dropout 0.3) and three output heads: building (3-class softmax), floor (5-class softmax) and coordinates (2-output linear regression). Training uses the Adam optimiser with a combined cross-entropy and mean-squared-error loss, weight decay and early stopping on a held-out 10% of the training scans. Hyperparameters are chosen by 5-fold GroupKFold cross-validation by user; the official validation file is used once as the test set.

Index Terms—indoor positioning, Wi-Fi fingerprinting, RSSI, multitask learning, multilayer perceptron, UJIIndoorLoc, dropout, early stopping

I. Introduction

Satellite positioning fails inside buildings because walls and ceilings block and reflect the signal. Many scenarios still need a location estimate: hospital asset tracking, campus navigation, emergency response. Almost every building is covered by Wi-Fi, and a phone can measure the received signal strength indicator (RSSI) of every access point (AP) it hears without connecting to any of them. Converting RSSI to position with a propagation model is unreliable indoors because of walls, furniture and multipath reflections [1]. Fingerprinting avoids this: a radio map of RSSI vectors is recorded at known positions offline, and a new scan is located by matching it to the radio map [2].

The task combines two classification problems (which building, which floor) and one regression problem (longitude and latitude in metres). Phase 1 of this project built a leakage-free machine learning pipeline on the UJIIndoorLoc dataset [3]. The main findings were as follows. (i) The "+100 = not detected" value must be replaced with a value below the minimum observed signal, otherwise it looks like the strongest possible reading. (ii) Cross-validation must hold out whole users, not random scans, to avoid leaking near-duplicate fingerprints. (iii) Logistic regression separates buildings almost perfectly (100% on the test set) because buildings use disjoint APs, but it struggles on floor. (iv) The per-building Random Forest reaches 91.3% floor accuracy and 9.37 m mean error on the official test set; k-NN reaches 8.69 m. (v) All Phase 1 models treat building, floor and position as three independent tasks. None of them shares what it learns about one target with the other two.

Phase 2 addresses this last gap. We train one multitask multilayer perceptron (MLP) with a shared trunk and separate output heads for each target. The trunk learns a single representation of the 465 AP signals that is useful for all three tasks simultaneously. Building and floor accuracy both depend on which APs are heard and how strongly, and so does position: sharing the trunk lets information flow between all three. This idea was formalised by Caruana [4] and has since been applied to indoor positioning by Kim et al. [5].

Our contributions in Phase 2 are:

- a multitask MLP with a shared trunk trained end-to-end on a combined loss;
- a regularisation sweep (input scaling, dropout, weight decay) on a fixed architecture;
- ablation studies on multitask vs. single-task training, loss weights and trunk size;
- a hybrid model that feeds trunk features into a per-building Random Forest;
- a comparison with the Phase 1 baselines on the identical split and metrics.

II. Related Work

Fingerprinting. RADAR [2] introduced Wi-Fi fingerprinting with nearest-neighbour matching in signal space. Horus [6] modelled the RSSI distribution at each location probabilistically, showing how noisy individual readings are and motivating learning-based methods.

UJIIndoorLoc. Torres-Sospedra et al. [3] released our dataset: the first large multi-building, multi-floor public benchmark, with a k-NN baseline reaching 89.9% floor accuracy and 7.9 m mean position error. The IPIN 2016

competition [7] compared many methods on building accuracy, floor accuracy and position error, the same three metrics we report.

Deep learning on UJIIndoorLoc. Nowicki and Wietrzykowski [8] trained a stacked autoencoder to compress the 520-dimensional RSSI vector, then classified building and floor with a feed-forward network, reaching about 92% combined building-and-floor accuracy with not-detected values replaced by -110 dBm and all inputs scaled together. Kim et al. [5] used a single network with an autoencoder front end for all three targets at once (99.82% building, 91.27% floor, 9.29 m), small enough to run on a phone. CNNLoc [9] combined an autoencoder with a one-dimensional CNN and reported 100% building accuracy, up to 96.03% floor accuracy, and 11.78 m mean position error in the setting with 94.68% floor accuracy, on the official test file.

Multitask learning. Caruana [4] showed that training related tasks with a shared hidden representation often reduces generalisation error compared with separate models. The shared layers cannot overfit to any single task's noise, and what they learn for one task carries over to the others through the shared weights (inductive transfer). Our shared trunk applies this idea directly: building, floor and coordinates are trained together so each task benefits from the signal in the others.

Regularisation. Dropout [10] randomly disables hidden units during training, preventing co-adaptation and reducing overfitting. Adam [11] adapts the learning rate for each weight individually, which is effective for sparse inputs like RSSI vectors. Weight decay [12] penalises large weights and improves generalisation. Prechelt [13] compared early-stopping rules and showed the trade-off between stopping early, which saves training time, and waiting longer for a slightly lower error.

Gaps addressed here. Prior deep learning work on UJIIndoorLoc either predicted only building and floor [8] or used a heavier autoencoder pre-training stage [5], [9]. We train a plain MLP trunk end-to-end with a combined loss, compare the result against the Phase 1 baselines on the same split and metrics, and test whether a hybrid model (trunk features feeding a Random Forest) adds anything over the network alone.

III. Dataset and Regularisation

A. Data and split

We use the same processed dataset as Phase 1. The training file contains 19,227 scans after removing 637 exact duplicates and 73 empty scans; the official validation file, collected four months later by different users on mostly different phones, serves as the test set (1,111 scans, used once at the very end). After dropping the 55 APs never heard in training and replacing the "+100 = not detected" value with -105 dBm (one below the minimum observed), each scan has 465 features. All statistics are fitted on training data and applied to the test set without refitting.

B. Input scaling

Neural networks multiply inputs by learned weights and sum them, so input scale affects gradient magnitudes. If inputs have very different ranges, some weights receive much larger gradient updates than others, which makes training unstable. We scale all 465 features to $[0, 1]$ with min-max scaling using the training minimum (-105) and maximum (0):

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} = \frac{x + 105}{105} \tag{1}$$

This maps the not-detected value exactly to 0 and the strongest observed signal to 1. We also tested standard scaling (subtract mean, divide by standard deviation per column) and powed scaling. Standard scaling performed much worse (around 20–24 m mean error in cross-validation against 11–12 m for min-max), because most columns are nearly constant at the not-heard value: dividing by their tiny standard deviation inflates the few scans where the AP is heard into enormous input values. Powed scaling sits in between. Min-max is kept.

C. Dropout

Dropout [10] randomly sets a fraction of hidden units to zero on each forward pass during training. The remaining units are scaled up so the expected output is unchanged. Units that are switched off in one batch cannot rely on specific partners to fix their errors; each unit must therefore learn a feature that is independently useful. At test time dropout is disabled and the full network is used. We apply dropout after every trunk layer with rate 0.3. Without any regularisation, mean position error in cross-validation is 12.40 m; dropout alone brings it to 11.76 m; combined with weight decay it falls to 11.17 m (see docs/dl_regularisation.md).

D. Weight decay

Weight decay adds a penalty proportional to the squared magnitude of every weight to the total loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \frac{\lambda}{2} \sum_{w} w^2 \tag{2}$$

with $\lambda = 10^{-4}$. During each update, this penalty pulls every weight slightly toward zero, preventing any single connection from dominating the prediction and smoothing the decision boundary. In PyTorch it is set via the weight_decay argument of the Adam optimiser.

E. Early stopping

Training stops when the validation loss has not improved for a fixed number of consecutive epochs (patience); the weights from the best epoch are then restored. We monitor a randomly sampled 10% of each fold's training scans with patience 30 and a maximum of 300 epochs. The random 10% split samples from all users and locations in proportion, so the validation loss it gives is

smooth enough to stop on. The choice of which scans to hold out for early stopping is discussed further in Section IV.

IV. Method

A. Architecture

The network has one shared trunk and three task-specific heads.

Input. A scaled vector $\mathbf{x} \in [0,1]^{465}$ of Wi-Fi RSSI readings.

Trunk. Three fully connected layers with ReLU activation and dropout (rate 0.3) after each layer:

$$\mathbf{h}^{(l)} = \mathrm{Dropout}\Big(\mathrm{ReLU}\Big(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\Big), p = 0.3\Big) \quad (3)$$

Layer sizes are 1024, 512, 256. The final 256-unit vector $\mathbf{h}^{(3)}$ is the trunk output, shared by all three heads. A single layer of 256 units gave 15.89 m in cross-validation; the three-layer 1024-512-256 trunk gave 11.08 m at no loss on floor accuracy, so it is used throughout (see docs/dl_ablations.md).

Heads. Three independent linear layers read from the trunk output:

- Building: $\hat{\mathbf{b}} = \mathrm{softmax}(\mathbf{W}_b\mathbf{h}^{(3)} + \mathbf{c}_b)$, three outputs.
- Floor: $\hat{\mathbf{f}} = \mathrm{softmax}(\mathbf{W}_f\mathbf{h}^{(3)} + \mathbf{c}_f)$, five outputs.
- Coordinates: $\hat{\mathbf{p}} = \mathbf{W}_p\mathbf{h}^{(3)} + \mathbf{c}_p$, two outputs (no activation).

Sharing the trunk means that gradients from all three heads flow back through the same weights. What the trunk learns about which APs identify a floor also informs position prediction and vice versa, the inductive transfer described by Caruana [4]. The ablation partly supports this: a multitask trunk reaches 11.08 m on position against 11.50 m for three separate single-task trunks of the same size, floor accuracy is a tie (82.9% vs. 83.2%), and building accuracy drops from 98.3% to 96.5% (docs/dl_ablations.md).

B. Loss function

The total loss is a weighted sum of three terms:

$$\mathcal{L} = \mathrm{CE}_b + \mathrm{CE}_f + w_{xy}\,\mathrm{MSE}_{xy} \quad (4)$$

where $\mathrm{CE}_b = -\log \hat{b}_c$ and $\mathrm{CE}_f = -\log \hat{f}_c$ are the cross-entropy losses for the true building class c and true floor class c, and MSE_{xy} is the mean squared error of the predicted coordinates.

Coordinate scaling. Raw UTM latitude in this dataset is around 4,864,000 and longitude around −7,500. An untrained network outputs values near 0, so its first coordinate errors would be millions of metres and their squares around 10^{13}, while each cross-entropy term is of order 1. Without scaling, the coordinate loss would swallow the gradient and the classification heads would barely train. We centre and scale the coordinates with the training mean and standard deviation before computing MSE_{xy}; predictions are unscaled back to metres for reporting.

Loss weight. We set $w_{xy} = 20$. With equal weights ($w_{xy} = 1$) the two cross-entropy terms dominate and the coordinate head is under-trained. In the tuning sweep (512-256 trunk, patience 30) weight 1 gives 13.83 m in cross-validation and weight 20 gives 12.52 m without hurting floor accuracy; weights 20 and 30 tie (12.52 vs. 12.48 m) so the simpler value is kept (docs/dl_architecture.md, Section 3). The ablation repeats the comparison with the final trunk: 13.35 m at weight 1 against 11.08 m at weight 20.

C. Training

We use the Adam optimiser [11] with learning rate 10^{-3}, weight decay 10^{-4} and batch size 256. Adam tracks a running mean and variance of the gradient for each weight and divides the update by the square root of the second moment. This gives each weight an individual effective step size, which is especially useful for sparse inputs: APs that are rarely heard contribute zero gradient in most batches and large gradients in the few batches where they appear. Training runs for up to 300 epochs per fold.

Early-stopping problem and fix. Our first implementation held out whole users for the early-stopping validation check, to match the outer 5-fold cross-validation. With only one or two users held out per fold, the validation loss was too noisy to stop on: its lowest value was at epoch 1 and no later epoch beat it by the minimum step, so after the 30 patience epochs the stopper restored the epoch-1 weights, which were barely trained. On one fold this produced 35% floor accuracy and 131 m mean position error, whatever the training length.

The fix is to use a random 10% sample of the training scans for the early-stopping check instead of whole users. This sample is drawn proportionally from all users and locations, so the validation loss is smooth enough to stop on. The same fold that failed with the user-based split reached 95% floor accuracy with the random split. The random 10% is entirely inside the current training fold, so no information from the outer validation users or from the test set is used. Best weights are restored at the end of each fold; on the final training run the network stopped at epoch 108 and best weights were from epoch 78.

D. Validation protocol

We use 5-fold GroupKFold with USERID as the group: no user appears in both the training and validation parts of a fold. This prevents near-duplicate scans (from the same walk at the same spot) from leaking between parts. All hyperparameters (trunk size, loss weights, dropout rate, weight decay, early-stopping patience) are chosen by mean position error in cross-validation on the training file only. The official validation file is used once, at the very end, as the test set after all choices are fixed. Using it at any earlier stage would be data leakage.

V. Results

VI. Discussion

VII. Conclusion and Future Work

Team Contributions

Rishi Raj	Network architecture, training loop, final settings, notebook, merging, report Sections V–VII
Aditya Samadhiya	Ablation studies, hybrid models
Yash Yadav	Input scaling and regularisation sweep
Kaustubh Ranjan Mishra	Literature review, theory notes, report Sections I–IV
Yash Raj	Final test run, error analysis, figures

Reproducibility

Code, figures and an executed notebook are in the project repository. Running python scripts/tune_network.py, compare_architectures.py, run_dl_regularisation.py, run_dl_ablations.py, run_hybrid.py and run_dl_analysis.py in order reproduces every Phase 2 result (random seed 42).

References

[1] F. Zafari, A. Gkelias, and K. K. Leung, "A survey of indoor localization systems and technologies," IEEE Communications Surveys & Tutorials, vol. 21, no. 3, pp. 2568–2599, 2019.

[2] P. Bahl and V. N. Padmanabhan, "RADAR: An in-building RF-based user location and tracking system," in Proc. IEEE INFOCOM, 2000, pp. 775–784.

[3] J. Torres-Sospedra, R. Montoliu, A. Martínez-Usó, J. P. Avariento, T. J. Arnau, M. Benedito-Bordonau, and J. Huerta, "UJIIndoorLoc: A new multi-building and multi-floor database for WLAN fingerprint-based indoor localization problems," in Proc. Int. Conf. Indoor Positioning and Indoor Navigation (IPIN), 2014, pp. 261–270.

[4] R. Caruana, "Multitask learning," Machine Learning, vol. 28, no. 1, pp. 41–75, 1997.

[5] K. S. Kim, S. Lee, and K. Huang, "A scalable deep neural network architecture for multi-building and multi-floor indoor localization based on Wi-Fi fingerprinting," Big Data Analytics, vol. 3, p. 4, 2018.

[6] M. Youssef and A. Agrawala, "The Horus WLAN location determination system," in Proc. ACM MobiSys, 2005, pp. 205–218.

[7] J. Torres-Sospedra et al., "The smartphone-based offline indoor location competition at IPIN 2016: Analysis and future work," Sensors, vol. 17, no. 3, p. 557, 2017.

[8] M. Nowicki and J. Wietrzykowski, "Low-effort place recognition with WiFi fingerprints using deep learning," in Automation 2017, Advances in Intelligent Systems and Computing, vol. 550. Springer, 2017.

[9] X. Song et al., "A novel convolutional neural network based indoor localization framework with WiFi fingerprinting," IEEE Access, vol. 7, pp. 110 698–110 709, 2019.

[10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, no. 1, pp. 1929–1958, 2014.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. on Learning Representations (ICLR), 2015.

[12] A. Krogh and J. A. Hertz, "A simple weight decay can improve generalization," in Advances in Neural Information Processing Systems 4, 1992, pp. 950–957.

[13] L. Prechelt, "Early stopping, but when?" in Neural Networks: Tricks of the Trade, ser. Lecture Notes in Computer Science. Springer, 1998, vol. 1524, pp. 55–69.